КЛАССЫ И ОБЪЕКТЫ

Объектно-ориентированное программирование (далее ООП) является основой всех разрабатываемых программных продуктов. Основой ООП служат классы и их объекты. В языке C#, который является полностью объектно-ориентированным все строится на классах и его объектах, раннее вы уже сталкивались с использованием объектов и классов, например создание массивов или вывода и ввода в консоль. Все эти элементы являются объектами или полноценными классами, так как в языке существуют классы, которым не требуется создание экземпляров.

ПОНЯТИЕ КЛАССА

Класс — пользовательский тип данных, который описывает данные, которые хранит объект, какие действия он совершает, как он создается и как может себя вести.

По сути, класс — это шаблон, на основе которого создаются реальные объекты. Для примера можно привести класс человека (Person). У любого человека есть имя, возраст, рост, вес и прочие данные (это будет являться полями класса). Однако у каждого человека эти данные разные, и каждый отдельный человек будет являться объектом, со своим именем, возрастом и т.д.

Отсюда можно сформулировать определение **объекта** — конкретный экземпляр класса, созданный в памяти программы во время её выполнения. Объект занимает определённое место в оперативной памяти, где хранятся значения его полей. В программе может существовать множество объектов класса, каждый будет иметь уникальное состояние.

Минимальная структура класса выглядит следующим образом:

```
class название_класса
{
   // содержимое класса
}
```

После слова class идет имя класса и далее в фигурных скобках идет собственно содержимое класса.

```
class Person
{
}
```

У класса есть ряд элементов:

1. Данные

   1.1.Поля — переменные, объявленные в классе

   1.2.Свойства — управление доступа к полям

   1.3.Константы — переменные, значение которых известно на этапе компиляции. Значение констант не может быть изменено

2. Поведение

   2.1.Конструкторы — специальные методы, которые вызываются при создании объекта

   2.2.Методы — блоки кода, выполняющие определенные операции

СОЗДАНИЕ КЛАССА

Для создания классов можно использовать несколько способов:

1. Создание и работа с классом в исполняемом файле Program.cs

Этот способ подразумевает создание класса и работу с его объектами в главном файле, где производятся все основные действия, но стоит помнить, что в современных версиях .NET используется подход top-level statements (инструкции верхнего уровня). Это означает, что файл Program.cs может не

содержать явного объявления метода Main, а код выполняется последовательно, сверху вниз, как в скриптовых языках.

Поэтому важно помнить, что объявления классов должны находиться после выполнения верхнеуровневого кода. Компилятор сначала обрабатывает исполняемые инструкции, а затем — объявления типов.

```
// файл Program.cs
Console.WriteLine("Программа запущена");

class Person // класс определяется после всех действий
{
}
```

Если попытаться объявить класс выше выполняемого кода — возникнет ошибка компиляции CS8803: "Top-level statements must precede namespace and type declarations."

2. Создание отдельного файла для класса

В данном способе используется модульный подход, то есть для каждого класса используется отдельный файл. На практике в профессиональной разработке классы почти всегда выносятся в отдельные файлы. Это соответствует принципу одна ответственность и значительно улучшает читаемость, сопровождаемость и навигацию по коду.

К правилам организации при таком подходе относятся:

– имя файла обычно совпадает с именем класса (например, Person.cs);

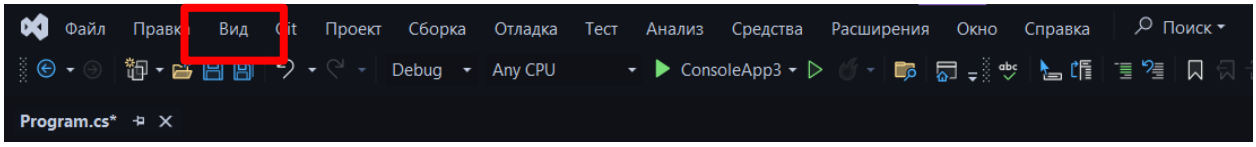
– в одном файле рекомендуется размещать один основной класс;

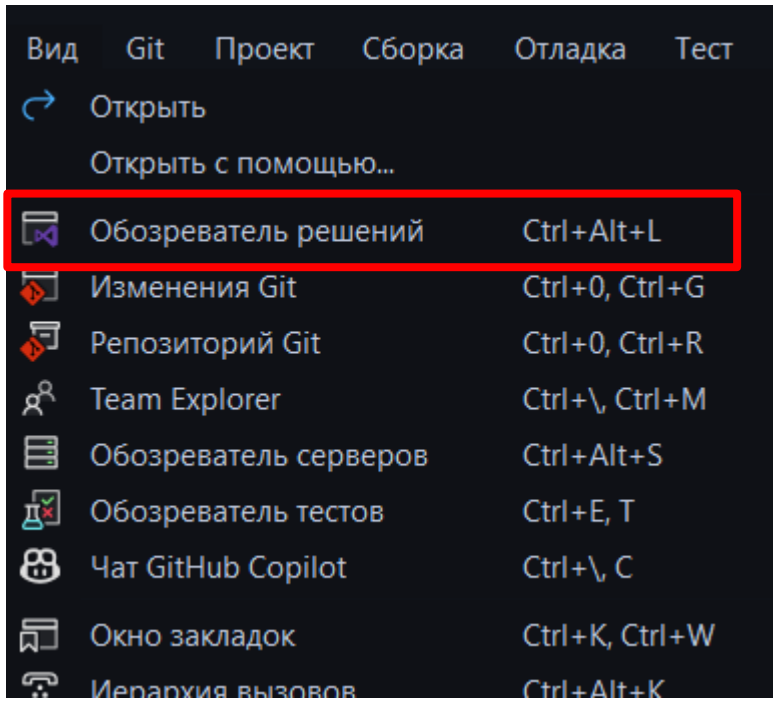
– файл автоматически подключается к проекту, не требуя дополнительных директив using (если находится в том же пространстве имён).

ИНСТРУКЦИЯ ДЛЯ СОЗДАНИЯ ФАЙЛА ДЛЯ КЛАССА

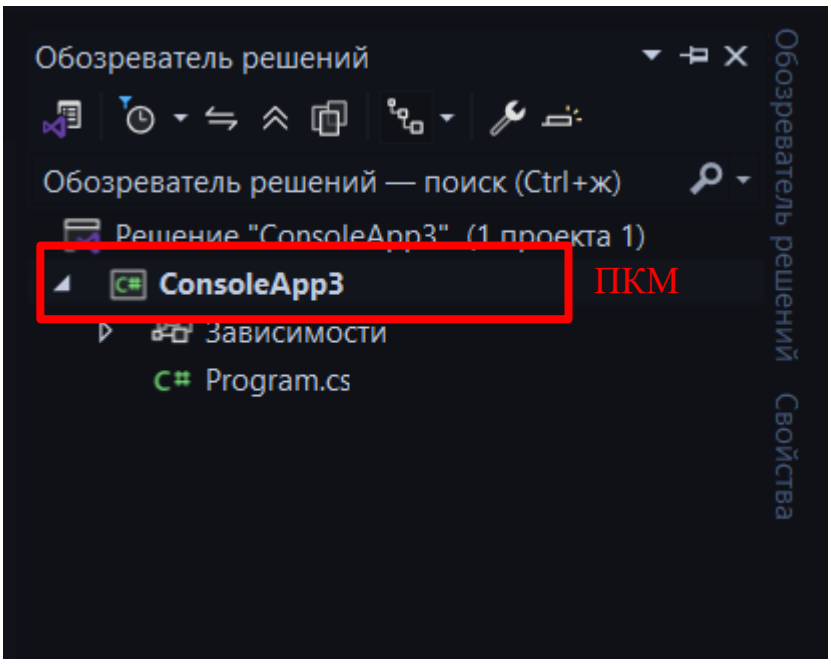
1. Откройте Visual Studio и создайте проект

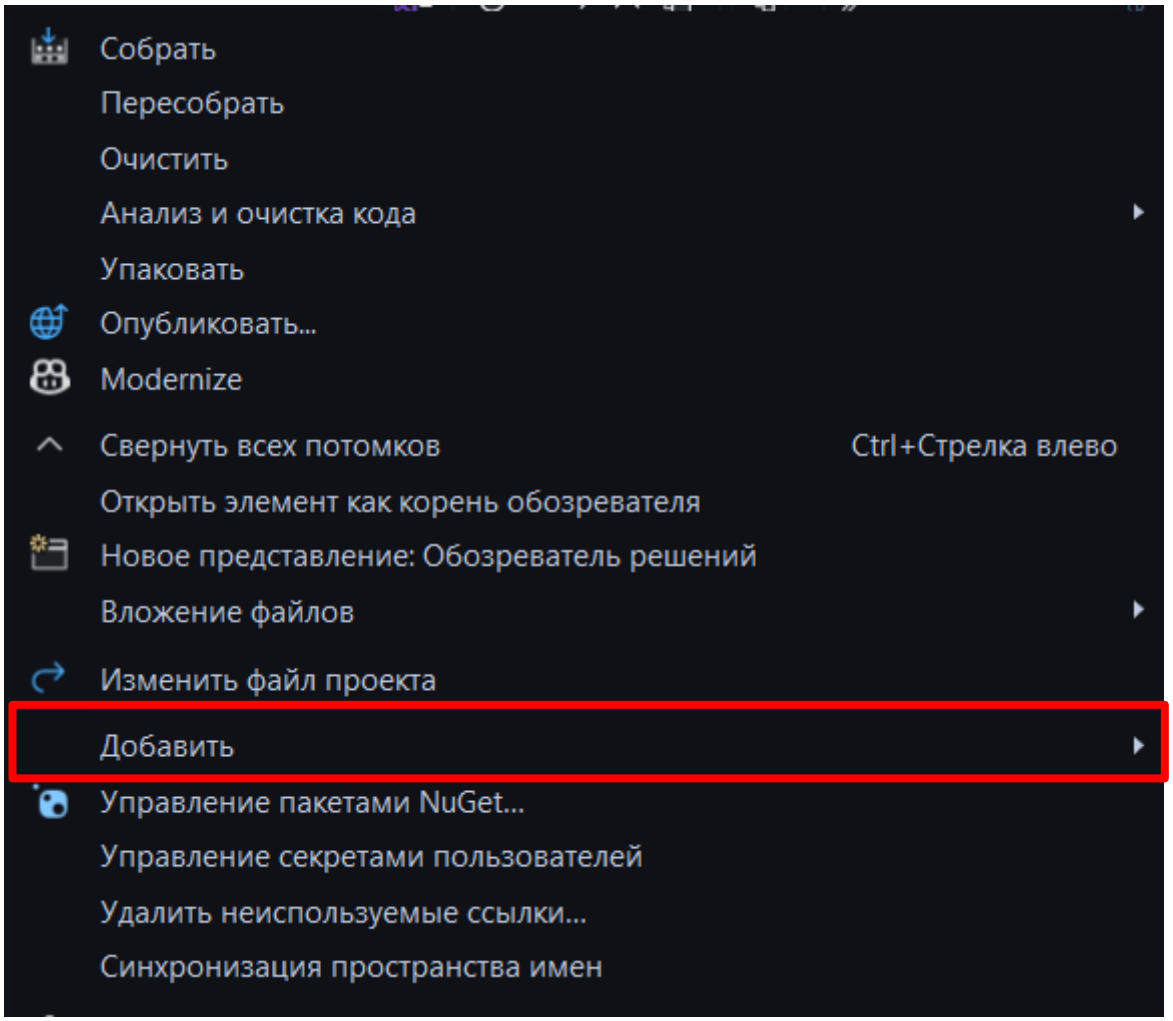
2. Справа находится «Обозреватель решений», он может быть как закреплен (то есть виден всегда), либо открепленен (необходимо нажать на него, чтобы раскрыть). Если же обозреватель решений отсутствует на рабочей области, то необходимо его добавить. Для этого сверху нажмите «Вид» → «Обозреватель решений», либо нажмите сочетание клавиш ctrl + alt + L

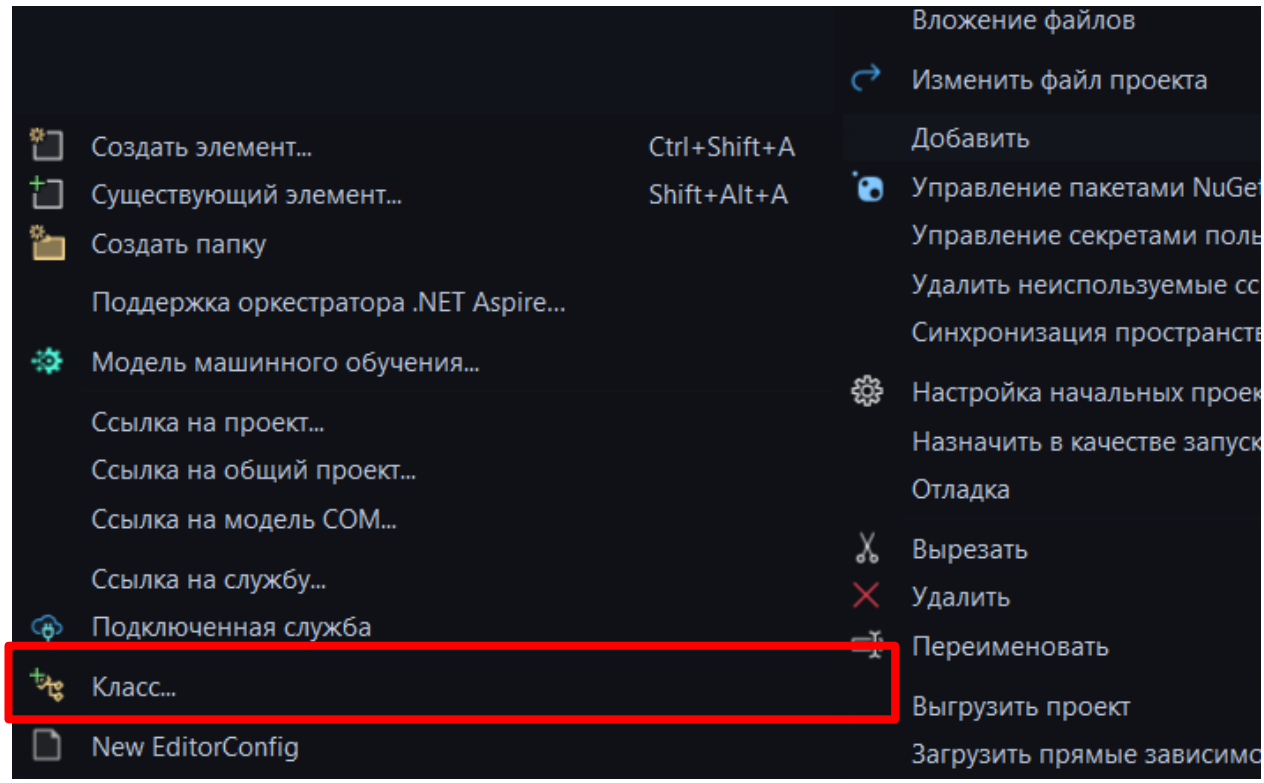




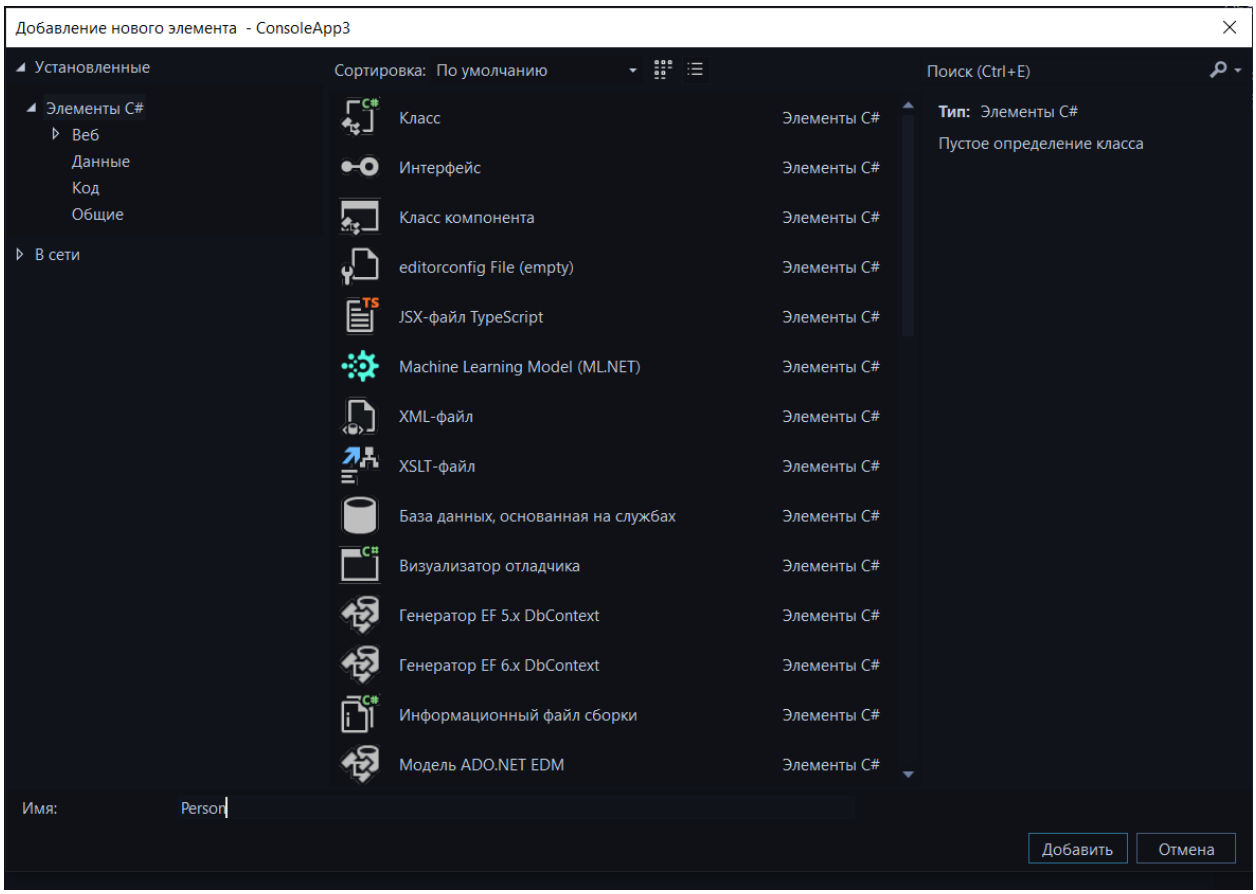
3. В обозревателе решений нажмите ПКМ (правая кнопка мыши) по файлу проекта, появится контекстное меню, на котором необходимо выбрать «Добавить» → «Класс»


Обозреватель решений
Обозреватель решений — поиск (Ctrl+ж)
Решение "ConsoleApp3" (1 проекта 1)
ConsoleApp3
ПКМ
Зависимости
Program.cs
Обозреватель решений
Свойства

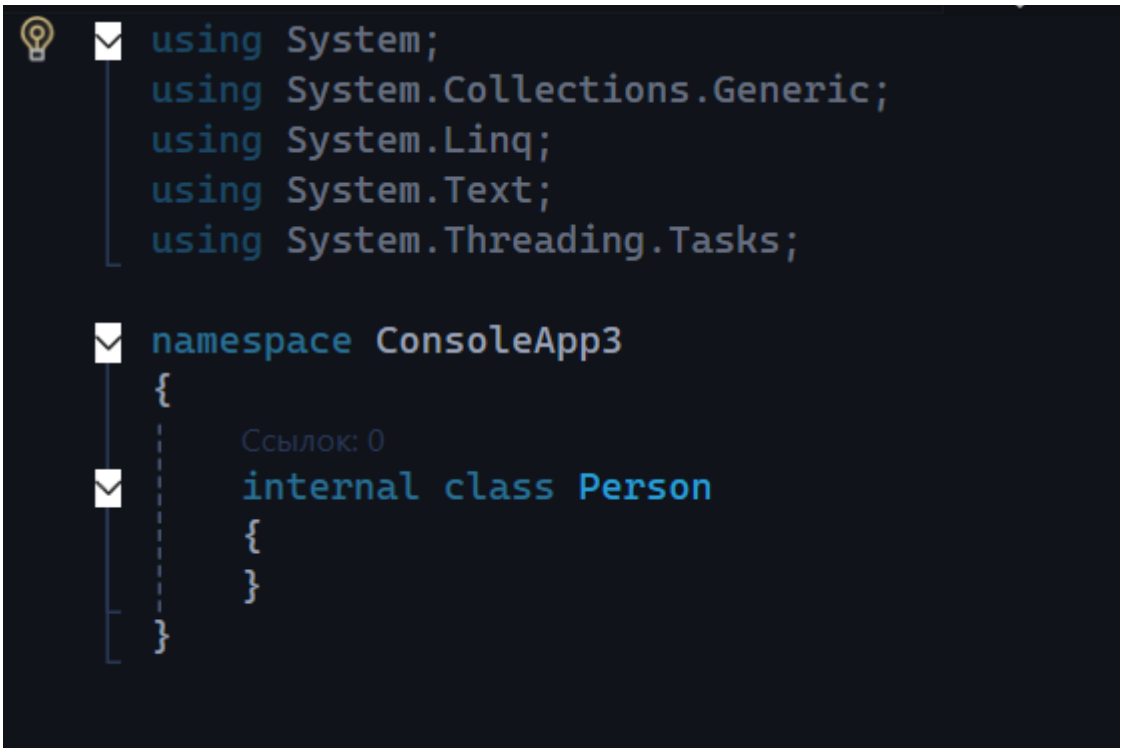

Собрать
Пересобрать
Очистить
Анализ и очистка кода
Упаковать
Опубликовать...
Modernize
Свернуть всех потомков
Ctrl+Стрелка влево
Открыть элемент как корень обозревателя
Новое представление: Обозреватель решений
Вложение файлов
Изменить файл проекта
Добавить
Управление пакетами NuGet...
Управление секретами пользователей
Удалить неиспользуемые ссылки...
Синхронизация пространства имен



4. Далее откроется окно, в котором нужно написать название файла, оно же будет являться названием класса. Для хорошего и правильно написанного кода важно: называть только на латинице, то есть использовать только английские буквы и названия, по соглашению наименований все названия классов пишутся с заглавной (большой) буквы.



5. После откроется файл с созданным классом



```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace ConsoleApp3
{
    Ссылок: 0
    internal class Person
    {
    }
}
```

МОДИФИКАТОРЫ ДОСТУПА

В понятии классов и объектов существуют также модификаторы доступа, которые определяют, откуда можно использовать класс или его элементы. Они реализуют принцип инкапсуляции — сокрытия внутренней реализации и предоставления строго определённого интерфейса для взаимодействия.

Наиболее распространённые модификаторы доступа:

Модификатор	Описание
public	элемент доступен отовсюду: из любого кода в текущей сборке и из других сборок, которые ссылаются на данную.
private	элемент доступен только внутри класса, где он объявлен. Это самый строгий модификатор, обеспечивающий максимальную инкапсуляцию.
protected	элемент доступен в классе, где объявлен, и во всех производных (наследующих) классах. Используется при построении иерархий наследования.
internal	элемент доступен внутри текущей сборки (проекта), но не извне. Полезен для скрытия служебной логики от потребителей библиотеки.

Поля класса

Поле — переменная, которая напрямую хранит состояние объекта. Поля определяют атрибуты или характеристики объекта.

```
class Person
{
   private string name;
   private int age;
   private bool isStudent;
}
```

Каждый объект Person будет иметь своё собственное имя, возраст и статус студента.

Если поле не инициализировано явно при объявлении или в конструкторе, оно получает значение по умолчанию в зависимости от своего типа:

– Целочисленные типы (int, long, byte и др.) → 0
– Типы с плавающей точкой (float, double, decimal) → 0.0
– bool → false
– char → '\0' (символ с кодом 0)
– string и любые другие ссылочные типы → null

СОГЛАШЕНИЕ ОБ ИМЕНОВАНИИ

C# использует строгие соглашения по именованию, которые являются частью общепринятых стандартов. Соблюдение этих правил делает код читаемым и понятным для других разработчиков.

PascalCase (заглавная буква каждого слова):

– Классы, структуры, интерфейсы, перечисления, делегаты (Person, StringBuilder, IEnumerable)
– Свойства, методы, события, константы (FirstName, CalculateTotal(), ValueChanged)
– Пространства имён (System.Collections.Generic)

camelCase (первое слово с маленькой буквы, остальные с заглавной):

– Параметры методов, локальные переменные (firstName, userAge, itemCount)
– private-поля
– Аргументы в анонимных методах и лямбда-выражениях

Современный стандарт для private-полей:

– private-поля используют camelCase без префиксов: private int age;

– Стиль с подчёркиванием (_age);

– Статические private-поля могут использовать s_ префикс: private static int s_count;

Общие правила:

– Имена должны быть осмысленными и описывать назначение

– Избегайте сокращений, кроме общепринятых (id, url, xml)

– Для булевых свойств/методов используйте префиксы Is, Can, Has: IsValid, CanRead, HasChildren

Свойства

Свойство — это контролируемый доступ к данным класса. В отличие от поля, которое просто хранит значение, свойство представляет собой пару методов (getter и setter), которые обеспечивают контролируемый доступ к значению. Свойства позволяют:

Контролировать чтение и запись — можно сделать свойство только для чтения (get) или только для записи (set)

– Добавлять валидацию — проверять корректность устанавливаемых значений

– Скрывать внутреннюю реализацию — менять способ хранения данных, не меняя публичный интерфейс

– Вычислять значения на лету — возвращать результат вычислений вместо хранимого значения

– Реализовывать логику уведомлений — вызывать события при изменении значения

Полное свойство явно определяет поле для хранения данных и методы доступа:

```
private int age; // Закрытое поле

public int Age
{
    get
    {
        return age; // Getter - возвращает значение
    }
    set
    {
        age = value; // Setter - устанавливает значение
        // value - ключевое слово, содержащее передаваемое значение
    }
}
```

Сокращённая форма

Можно также использовать сокращённый синтаксис для свойств с простой логикой:

```
public int Age
{
    get => age;
    set => age = value;
}
```

Один из ключевых примеров полезности свойств — добавление проверок:

```
public int Age
{
    get => age;
    set
    {
        if (value < 0)
        {
            throw new ArgumentException("Возраст не может быть отрицательным");
        }
```

```
            else if (value > 150)
            {
                age = 150;
            }
            else
            {
                age = value;
            }

        }
    }
```

Автосвойства

Автосвойства позволяет объявить свойство без явного определения поля:

```
public string Name { get; set; }
```

Компилятор сам создаёт скрытое поле и стандартные getter/setter. Автосвойства используются, когда не требуется дополнительная логика при получении или установке значения.

Разновидности автосвойств

```
public string Name { get; set; } = "Unknown"; // С инициализацией
public int Id { get; } // Только для чтения (устанавливается в конструкторе)
```

Различия между полями, свойствами и автосвойствами:

Поле — примитивное хранилище данных, обычно private

Свойство — контролируемый доступ к данным с возможностью добавления логики

Автосвойство — упрощённая форма свойства, где логика доступа тривиальна

КОНСТРУКТОРЫ КЛАССА

Конструктор — это специальный метод, который вызывается автоматически при создании нового экземпляра класса с помощью оператора new. Его основная задача — инициализировать объект, установить начальные значения полей, выделить ресурсы.

Имя конструктора всегда совпадает с именем класса, и у него нет возвращаемого типа (даже void).

Конструктор по умолчанию

Если в классе не объявлен ни один конструктор, компилятор автоматически создаёт конструктор по умолчанию (без параметров), который не делает ничего особенного:

```
public Person()
{
   // Пустое тело, поля получают значения по умолчанию
}
```

Можно явно определить конструктор по умолчанию с дополнительной логикой:

```
public Person()
{
   age = 0;
   name = "Unknown";
}
```

Конструктор с параметрами

Конструкторы могут принимать параметры для инициализации объекта конкретными значениями:

```
public Person(string name, int age)
{
   this.name = name;
   this.age = age;
}
```

Ключевое слово this

this — это ссылка на текущий экземпляр класса.

Используется:

– Для разрешения конфликта имён между параметрами и полями класса

– Для передачи текущего объекта в качестве аргумента в другие методы

– Для вызова других конструкторов того же класса (цепочка конструкторов)

Перегрузка конструкторов

Класс может иметь несколько конструкторов с разными наборами параметров:

```
public Person() : this("Unknown", 0)
{
    // Дополнительная инициализация, если нужна
}

public Person(string name) : this(name, 0) // Вызов основного конструктора
{
}

public Person(string name, int age) // Основной конструктор
{
    this.name = name;
    this.age = age;
}
```

МЕТОДЫ КЛАССА

Методы описывают поведение объекта — действия, которые объект может выполнять. Если поля и свойства определяют «чем объект является», то методы определяют «что объект умеет делать».

```
public void SayHello()
{
   Console.WriteLine($"Меня зовут {Name}, мне {Age} лет.");
}
```

Методы могут возвращать результат своей работы:

```
public int GetBirthYear(int currentYear)
{
   return currentYear - Age; // Возвращаем вычисленное значение
}

int birthYear = person.GetBirthYear(2024);
```

Сокращённая форма методов

Для методов, состоящих из одного выражения, можно использовать сокращённый синтаксис:

```
public bool IsAdult()
{
   return Age >= 18;
}

public bool IsAdult() => Age >= 18;
```

Перегрузка методов

Как и конструкторы, обычные методы можно перегружать — создавать несколько методов с одинаковым именем, но разными параметрами:

```
public void Introduce()
{
   Console.WriteLine($"Я {Name}");
}

public void Introduce(string greeting)
{
   Console.WriteLine($"{greeting}, я {Name}");
}
```

```
public void Introduce(string greeting, bool includeAge)
{
   if (includeAge)
      Console.WriteLine($"{greeting}, я {Name}, мне {Age} лет");
   else
      Console.WriteLine($"{greeting}, я {Name}");
}
```

КЛЮЧЕВОЕ СЛОВО STATIC

static означает принадлежность самому классу, а не конкретному экземпляру класса. Статические элементы существуют в единственном экземпляре для всего приложения и доступны без создания объекта.

Статическое поле

Статическое поле является общим для всех экземпляров класса:

```
public static int Population;

public Person()
{
   Population++;
}
```

Статический метод

Статические методы могут работать только со статическими членами класса:

```
public static void ShowPopulation()
{
   Console.WriteLine($"Всего создано людей: {Population}");
   // Console.WriteLine(Name); // ОШИБКА: нельзя обратиться к нестатическому члену
}
```

Вызов статических методов

```
Person.ShowPopulation();

// Через экземпляр тоже можно, но не рекомендуется
Person p = new Person();
p.ShowPopulation(); // Работает, но выдает предупреждение
```

Статический конструктор

Статический конструктор выполняется один раз при первом обращении к классу

```
static Person()
{
   Population = 0;
   Console.WriteLine("Статический конструктор Person вызван");
}
```

Статический класс

Статический класс не может иметь экземпляров и содержит только статические элементы:

```
public static class MathHelper
{
   public static double PI = 3.14159;

   public static int Add(int a, int b) => a + b;
   public static int Multiply(int a, int b) => a * b;
}


double circleArea = MathHelper.PI * radius * radius;
int sum = MathHelper.Add(5, 3);
```

Особенности статического класса:

- нельзя создать объект (нельзя использовать new)
- не может иметь нестатических членов
- не может быть унаследован
- часто используется для утилитных функций и расширяющих методов

ОБЪЕКТЫ КЛАССА

Объекты создаются с помощью оператора new, который:

- Выделяет память для объекта
- Инициализирует поля значениями по умолчанию
- Вызывает конструктор

```
// Создание с конструктором по умолчанию
Person p1 = new Person();

// Создание с конструктором с параметрами
Person p2 = new Person("Анна", 20);

// Использование var для вывода типа
var p3 = new Person("Петр", 25);
```

Инициализация через фигурные скобки

Позволяет установить значения свойств сразу после создания объекта

```
Person p4 = new Person
{
   Name = "Иван",
   Age = 30,
   IsStudent = true
};

// Эквивалентно:
Person p4 = new Person();
p4.Name = "Иван";
p4.Age = 30;
p4.IsStudent = true;
```

Изменение данных объекта

После создания объекта можно изменять его состояние через свойства и методы

```
p1.Name = "Новое имя"; // Используем setter свойства
p1.Age = p1.Age + 1;   // Увеличиваем возраст на 1

// Через метод
p1.UpdateName("Ещё одно имя");
```

Обращение к методам объекта

Методы вызываются для конкретного объекта и работают с его данными

```
p1.SayHello(); // Выведет информацию о p1
p2.SayHello(); // Выведет информацию о p2

// Методы с возвращаемым значением
bool isAdult = p1.IsAdult();
int birthYear = p1.GetBirthYear(2024);
```

Работа с несколькими объектами

Каждый объект независим и имеет собственное состояние:

```
Person alice = new Person("Алиса", 25);
Person bob = new Person("Боб", 30);

alice.Age = 26; // Изменяем только возраст Алисы
Console.WriteLine(bob.Age); // Выведет 30, возраст Боба не изменился
```

Уничтожение объектов и сборка мусора

В C# не нужно явно удалять объекты. Когда на объект больше нет ссылок, он становится кандидатом на удаление сборщиком мусора (Garbage Collector), который освободит занимаемую им память. В более низкоуровневых языках, где нет сборщика мусора присутствует необходимость использовать деструкторы, которые очистят память после вызова объекта и его работы.